❷ Fidé : Logiciel de gestion des résultats dans les tournois d'échecs

On s'intéresse ici au logiciel Fidé qui permet de gérer les résultats des différents tournois d'échecs qui sont organisés sous l'égide de la Fédération Internationale Des Echecs. Ce logiciel repose sur une base de données Oracle dont le schéma relationnel vous est communiqué ci-dessous.

LIGUES (idLigue, nomLigue, zoneLigue, *nbJoueursLigue*)
CLUBS (idClub, nomClub, idLigue#)
JOUEURS (idJoueur, nomJoueur, prenomJoueur, sexeJoueur, dateNaissanceJoueur, eloJoueur, idClub#)
TOURNOIS (idTournoi, nomTournoi, lieuTournoi, nbRondesTournoi, *nbParticipantsTournoi*)
PARTICIPER (idTournoi#, idJoueur#)
PARTIES (idPartie, idTournoi#, numRonde, numTable, idJoueurBlancs#, idJoueurNoirs#, resultatPartie)

La table *Clubs* recense les clubs dans lesquels sont licenciés les joueurs d'échecs. Chaque club est affilié à une ligue régionale (par exemple la ligue du Languedoc, la ligue d'Aquitaine, la ligue d'Ile de France, etc.). Dans la table *Ligues*, on peut trouver toutes les informations qui concernent ces ligues.

La table *Joueurs* contient les informations nominatives qui concernent les différents joueurs licenciés à la Fédération Internationale Des Echecs. Dans cette table, l'attribut `eloJoueur` permet de connaître le classement Elo du joueur. Ce classement, qui est un nombre compris entre 1 000 et 3 000, renseigne sur le niveau du joueur. Plus ce nombre est élevé, plus le joueur est fort – 1 600 : Pousseur de bois ; 1 800 : Mazette ; 2 000 : Bon joueur de club ; 2 200 : Patate Internationale ; 2 300 : Maitre de la Fédération Internationale ; 2 400 : Maitre International ; 2 500 : Grand Maitre International. Chaque joueur est licencié à un et un seul club (et donc rattaché à une seule ligue régionale).

Dans la table *Tournois* on trouve tous les tournois qui sont organisés par la Fédération Internationale des Echecs. Dans cette table, l'attribut `nbRondesTournoi` indique le nombre de rondes (c'est-à-dire de tours, ou de parties – round en anglais) que vont jouer chacun des participants pendant le tournoi. La table *Participer* permet de connaître les joueurs qui ont participé aux différents tournois.

Enfin, la table *Parties* stocke les résultats de toutes les parties qui ont été jouées par les joueurs dans les tournois. Dans cette table l'attribut `idTournoi` indique dans quel tournoi a été jouée la partie. `numRonde` permet de connaître la ronde (ou le tour) à laquelle cette partie a été jouée dans le tournoi ; par exemple, si `numRonde = 1`, cela veut dire que cette partie correspond à la première partie jouée par les joueurs dans le tournoi. Si `numRonde = 2`, il s'agit de la deuxième partie jouée par les joueurs dans le tournoi, etc.
L'attribut `idJoueurBlancs` indique l'identifiant du joueur qui avait les blancs dans la partie. Et `idJoueurNoirs` celui qui avait les noirs. `resultatPartie` donne le résultat de la partie. Il a pour valeur "1-0" si c'est les blancs qui ont gagné (le joueur avec les blancs marque alors 1 point et le joueur avec les noirs 0). "1/2" si la partie est déclarée nulle (les deux joueurs marquent un demi-point). Et "0-1" si c'est les noirs qui ont gagné (le joueur avec les blancs marque 0 point et le joueur avec les noirs 1).

Vous trouverez sur l'ENT le fichier « FIDE.sql » qui vous permettra de générer les tables de la base de données sur votre schéma.

Dans les tables *Ligues* et *Tournois* figurent respectivement les attributs `nbJoueursLigue` et `nbParticipantsTournoi` qui sont indiqués en italique dans le schéma relationnel. Ces attributs sont dits dérivés (au sens UML), c'est-à-dire qu'ils sont calculables à partir de requêtes SQL. Ils engendrent donc de la redondance dans la base de données mais ont tout de même été maintenus car leur contenu est relativement stable. Toutefois, dans le script « FIDE.sql » leur valeur n'a pas encore été initialisée, et sur toutes les lignes, ces attributs ont pour le moment la valeur `NULL`.

Première partie – Les Blocs PL/SQL :

1) Bloc PL/SQL simple et paquetage `DBMS_OUTPUT`.
   - Ecrire un bloc PL/SQL qui, grâce au paquetage `DBMS_OUTPUT`, affiche le nombre de joueurs qui appartiennent au club qui a l'identifiant 'C1'.

     On rappelle que pour que le paquetage `DBMS_OUTPUT` fonctionne dans l'interface iSQL*Plus, il faut l'activer avec la commande `SET SERVEROUTPUT ON` avant la première instruction du bloc PL/SQL (cette option sera valable durant toute la session iSQL*Plus).
   - Résultat attendu :

```
Il y a 6 joueur(s) dans le club C1
```

2) Utilisation de variables.
   - On souhaite maintenant tester le bloc PL/SQL de la question précédente avec d'autres clubs (par exemple les clubs C2, C3, ..., C9, ...). Mais on veut modifier un minimum de lignes de code lorsque on change le club, et ce grâce à l'utilisation d'une variable.

     Si cela n'a pas été fait lors de la question précédente, refactorer le code du bloc PL/SQL afin qu'il ne soit utile de modifier qu'une seule ligne de code lorsque on teste un autre club.
   - Résultats attendus :

```
Il y a 6 joueur(s) dans le club C1

Il y a 4 joueur(s) dans le club C2

Il y a 0 joueur(s) dans le club C9
```

3) Les Exceptions : l'exception `NO_DATA_FOUND`.
   - Dans la question précédente il y a un léger problème. En effet, si l'identifiant du club n'existe pas (par exemple le C11), il est indiqué qu'il y a 0 joueurs dans le club en question. Et on aurait eu la même réponse si on avait indiqué l'identifiant d'un club qui existe bien mais qui n'a encore aucun joueur licencié (par exemple le club C9).

     En s'inspirant du bloc PL/SQL écrit lors de la question précédente, écrire un nouveau bloc qui indiquerait le cas échéant qu'aucun club ne possède l'identifiant qui a été indiqué.

     On utilisera ici l'exception prédéfinie `NO_DATA_FOUND` qui est levée automatiquement lorsqu'une requête `SELECT … INTO` ne retourne rien (il est à noter que si la requête retourne plusieurs lignes, c'est alors l'exception `TOO_MANY_ROWS` qui est levée).
   - Résultats attendus :

```
Il y a 1 joueur(s) dans le club C10

Il y a 0 joueur(s) dans le club C9

Il n'y a pas de club C11
```

4) Structures de contrôle : la conditionnelle (`IF … THEN … ELSE … END IF`).
   - Ecrire un nouveau bloc PL/SQL qui fait exactement la même chose que la question précédente mais en utilisant cette fois-ci la structure de contrôle (`IF … THEN … ELSE … END IF`) à la place de la section `EXCEPTION`.
   - Résultats attendus :

```
Il y a 1 joueur(s) dans le club C10

Il y a 0 joueur(s) dans le club C9

Il n'y a pas de club C11
```

5) Les variables %ROWTYPE.
   - Ecrire un bloc PL/SQL qui permet d'afficher les informations de la table *Tournois* qui concernent le tournoi T1.

     Pour se simplifier la tâche, et pour faciliter les éventuelles futures maintenances de l'application, on récupèrera les données qui concernent le tournoi T1 dans une variable %ROWTYPE.

   - Résultat attendu :

```
Identifiant du tournoi : T1
Nom du tournoi : Tournoi de Noël
Lieu du tournoi : Montpellier
Nombre de rondes du tournoi : 5
```

Deuxième partie – Les Procédures et Fonctions stockées :

Fonctions stockées

6) Fonction stockée nbJoueursParClub.
   - En vous inspirant de ce que vous avez écrit lors des questions 3 ou 4, écrire une fonction stockée nbJoueursParClub qui retourne le nombre de joueurs qui appartiennent au club dont l'identifiant p_idClub est passé en paramètre. Cette fonction doit avoir la signature suivante :
     `FUNCTION nbJoueursParClub(p_idClub IN Clubs.idClub%TYPE) RETURN NUMBER`
     Si on passe en paramètre de cette fonction un identifiant de club qui n'existe pas, on souhaite que la fonction retourne NULL (et non pas 0).

     Si lors de la création de votre fonction, vous avez une erreur de compilation (Warning: Function created with compilation errors) vous pouvez demander à Oracle de vous afficher la liste des erreurs grâce à l'instruction SHOW ERRORS.

     On rappelle que sous Oracle, pour tester une fonction stockée, on peut utiliser la pseudo-table DUAL.

   - Résultats attendus :

```
SELECT nbJoueursParClub('C1')
FROM DUAL ;

nbJoueursParClub('C1')
----------------------
6

SELECT nbJoueursParClub('C9')
FROM DUAL ;

nbJoueursParClub('C9')
----------------------
0

SELECT nbJoueursParClub('C11')
FROM DUAL ;

nbJoueursParClub('C11')
-----------------------

// quand le résultat est NULL, par défaut, iSQL*Plus n'affiche rien
```

7) Fonction stockée nbJoueursParLigue.
   - Ecrire une fonction stockée nbJoueursParLigue qui retourne le nombre de joueurs qui sont rattachés (ou dont le club est rattaché) à une ligue dont l'identifiant p_idLigue est passé en paramètre. Cette fonction doit avoir la signature suivante :
     `FUNCTION nbJoueursParLigue(p_idLigue IN Ligues.idLigue%TYPE) RETURN NUMBER`

     Dans le corps de cette fonction, on utilisera la fonction qui a été réalisée à la question précédente (nbJoueursParClub).

On ne vous demande pas de gérer explicitement le cas où l'identifiant de la ligue qui est passé en paramètre n'existe pas, ou bien le cas où la ligue existe mais qu'elle n'a pas de club (dans ces cas-là, si vous utilisez dans votre code la fonction `SUM`, votre fonction devrait retourner `NULL` par défaut sans que vous soyez obligé de le programmer).

- Résultat attendu :

```
SELECT nbJoueursParLigue('LAN')
FROM DUAL ;

nbJoueursParLigue('LAN')
------------------------
14
```

- Ecrire une requête SQL qui, en utilisant la fonction écrite ci-dessus, met à jour l'attribut `nbJoueursLigue` de toutes les lignes de la table *Ligues*.
  Puis, vérifier que dans la table *Ligues* on a bien 14 joueurs dans la ligue du Languedoc.

```
SELECT nbJoueursLigue
FROM Ligues
WHERE nomLigue = 'Languedoc' ;

nbJoueursLigue
---------------
14
```

Procédures stockées

8) Procédure stockée affichageInfosTournoi.
   - En vous inspirant de ce que vous avez écrit lors de la question 5, écrire une procédure stockée `affichageInfosTournoi` qui affiche les informations qui concernent le tournoi dont l'identifiant `p_idTournoi` est passé en paramètre. Cette procédure doit avoir la signature suivante :

```
PROCEDURE affichageInfosTournoi(
                                p_idTournoi IN Tournois.idTournoi%TYPE)
```

     On rappelle que sous Oracle, on peut appeler une procédure stockée avec l'instruction `CALL`.

   - Résultat attendu :

```
CALL affichageInfosTournoi('T1');
Identifiant du tournoi : T1
Nom du tournoi : Tournoi de Noël
Lieu du tournoi : Montpellier
Nombre de rondes du tournoi : 5

CALL affichageInfosTournoi('T4');
Le tournoi T4 n'existe pas
```

9) Procédure stockée miseAJourNbParticipantsTournoi.
   - Ecrire une procédure stockée qui met à jour l'attribut `nbParticipantsTournoi` de toutes les lignes de la table *Tournois*. Pour connaître les participants à un tournoi, on pourra interroger la table *Participer*. Votre procédure devra donc avoir une requête `UPDATE` qui pourra contenir une requête imbriquée corrélée. Cette procédure doit avoir la signature suivante :

```
PROCEDURE miseAJourNbParticipantsTournoi
```

   - Résultat attendu :

```
CALL miseAJourNbParticipantsTournoi() ;

SELECT idTournoi, nomTournoi, nbParticipantsTournoi
FROM Tournois;

idTournoi  nomTournoi          nbParticipantsTournoi
---------- ------------------- ----------------------
T1         Tournoi de Noël     10
T2         Tournoi des Normes  16
T3         Open JCL            14
```

Troisième partie – Les Curseurs :

10) Procédure stockée affichageParticipantsTournoi.

- Ecrire une procédure stockée `affichageParticipantsTournoi` qui affiche tous les joueurs qui ont participé à un tournoi dont l'identifiant `p_idTournoi` est passé en paramètre. Il faut que les joueurs soient classés par ordre Elo décroissant (si plusieurs joueurs ont le même Elo, ils doivent être classés par rapport à l'ordre alphabétique de leur nom). Cette procédure doit avoir la signature suivante :

```
PROCEDURE affichageParticipantsTournoi(
                                p_idTournoi IN Tournois.idTournoi%TYPE)
```

- Résultat attendu :

```
CALL affichageParticipantsTournoi('T1') ;

Triser   Jessica 2385
Némard   Jean    2235
Alizan   Gaspard 2100
Palleja  Xavier  2080
Nanas    Judas   2030
Assin    Marc    1960
Sticko   Judas   1860
Bono     Jean    1850
Kaécouté Xavier  1850
Gator    Ali     1750
```

11) Procédure stockée affichageToutTournoi.

- Ecrire une procédure stockée `affichageToutTournoi` qui affiche toutes les informations qui concernent un tournoi dont l'identifiant `p_idTournoi` est passé en paramètre ; avec la liste des joueurs qui participent au tournoi. Cette procédure doit avoir la signature suivante :

```
PROCEDURE affichageToutTournoi(
                                p_idTournoi IN Tournois.idTournoi%TYPE)
```

- Résultat attendu :

```
CALL affichageToutTournoi ('T2') ;

Identifiant du tournoi : T2
Nom du tournoi : Tournoi de Normes
Lieu du tournoi : Vendargues
Nombre de rondes du tournoi : 7

Liste des participants :
Terrieur Alex    2455
Triser   Jessica 2385
Némard   Jean    2235
Oraque   Anne    2210
Zeblouse Agathe  2150
Alizan   Gaspard 2100
Nanas    Judas   2030
Terrieur Alain   2010
Micoton  Mylène  1990
Assin    Marc    1960
Vesselle Aude    1940
Outan    Laurent 1920
Stické   Sophie  1890
Bono     Jean    1850
Kaécouté Xavier  1850
Gator    Ali     1750
```

12) Procédure stockée affichageJoueursParLigueEtClub.

- Ecrire une procédure stockée `affichageJoueursParLigueEtClub` qui affiche pour tous les clubs qui appartiennent à une ligue dont l'identifiant `p_idLigue` est passé en paramètre, la liste des licenciés du club avec pour chacun d'entre eux le nombre de tournois auxquels ils ont participé. On veut que les clubs affichés soient classés par ordre alphabétique de leur nom. Et que dans un club, les joueurs soient classés par rapport au nombre de tournois auxquels ils ont participé (si plusieurs joueurs ont participé au même nombre de tournois, ils doivent être classés par rapport à leur Elo). Cette procédure doit avoir la signature suivante :

```
PROCEDURE affichageJoueursParLigueEtClub(
                                    p_idLigue IN Ligues.idLigue%TYPE)
```

- Résultat attendu :

```
CALL affichageJoueursParLigueEtClub('LAN');

Club : C2 Lattes Echecs (4 joueurs)
-----> Kaécouté Xavier 1850 a participé à 3 tournoi(s)
-----> Gator Ali 1750 a participé à 3 tournoi(s)
-----> Assin Marc 1960 a participé à 2 tournoi(s)
-----> Sticko Judas 1860 a participé à 1 tournoi(s)

Club : C1 Montpellier Chess Club (6 joueurs)
-----> Némard Jean 2235 a participé à 3 tournoi(s)
-----> Bono Jean 1850 a participé à 3 tournoi(s)
-----> Triser Jessica 2385 a participé à 2 tournoi(s)
-----> Alizan Gaspard 2100 a participé à 2 tournoi(s)
-----> Palleja Xavier 2080 a participé à 2 tournoi(s)
-----> Nanas Judas 2030 a participé à 2 tournoi(s)

Club : C3 Nimes Crocoéchecs (4 joueurs)
-----> Bricot Judas 1780 a participé à 1 tournoi(s)
-----> Croque Odile 1690 a participé à 1 tournoi(s)
-----> Javel Aude 1450 a participé à 1 tournoi(s)
-----> Niohrangina Nicolas 1570 a participé à 0 tournoi(s)



CALL affichageJoueursParLigueEtClub('IDF');

Club : C5 Grand Roque Paris (2 joueurs)
-----> Oraque Anne 2210 a participé à 1 tournoi(s)
-----> Timettre Vincent 1850 a participé à 1 tournoi(s)

Club : C4 Paris Chess XV (2 joueurs)
-----> Terrieur Alex 2455 a participé à 2 tournoi(s)
-----> Micoton Mylène 1990 a participé à 2 tournoi(s)



CALL affichageJoueursParLigueEtClub('MPY');

Club : C7 To-lose Fun Chess (4 joueurs)
-----> Vesselle Aude 1940 a participé à 1 tournoi(s)
-----> Sticé Sophie 1890 a participé à 1 tournoi(s)
-----> Ouzy Jacques 1840 a participé à 1 tournoi(s)
-----> Deuf John 1980 a participé à 0 tournoi(s)
```

Quatrième partie – Edition des résultats d'une ronde :

Dans cette partie, on souhaite pouvoir éditer les résultats complets d'une ronde d'un tournoi.

13) Fonction stockée categorieJoueur.

- Ecrire une fonction stockée `categorieJoueur` qui retourne la catégorie d'un joueur dont la date de naissance et le sexe sont passés en paramètres. Cette fonction doit avoir la signature suivante :

```
FUNCTION categorieJoueur(
              p_dateNaissance IN Joueurs.dateNaissanceJoueur%TYPE,
              p_sexe IN Joueurs.sexeJoueur%TYPE) RETURN VARCHAR
```

  La catégorie d'un joueur peut être SenM (pour Sénior Masculin), SenF (Sénior Féminine), VetM (Vétéran Masculin), VetF (Vétérane Féminine), JunM (Junior Masculin) ou JunF (Junior Féminine).
  Un joueur est Sénior s'il est né entre 1972 (inclus) et 2006 (inclus). Il est Junior s'il est né après 2006 et il est Vétéran s'il est né avant 1972.

- Résultat attendu :

```
SELECT categorieJoueur('1/1/1972', 'M')
FROM DUAL ;

categorieJoueur('1/1/1972', 'M')
---------------------------------
SenM

SELECT categorieJoueur('31/12/1971', 'F')
FROM DUAL ;

categorieJoueur('31/12/1971', 'F')
---------------------------------
VetF

SELECT categorieJoueur(dateNaissanceJoueur, sexeJoueur)
FROM Joueurs
WHERE idJoueur = 'J7';

categorieJoueur(dateNaissanceJoueur, sexeJoueur)
-----------------------------------------------
JunM
```

14) Procédure stockée affichageResultatsRonde.

- Ecrire une procédure stockée `affichageResultatsRonde` qui affiche tous les résultats qui ont eu lieu à la ronde `p_idRonde` du tournoi `p_idTournoi`. Cette procédure doit avoir la signature suivante :

```
PROCEDURE affichageResultatsRonde(
                            p_idTournoi IN Parties.idTournoi%TYPE,
                            p_numRonde IN Parties.numRonde%TYPE)
```

  On souhaite que les parties de la ronde soient ordonnées par rapport au numéro de la table.

- Résultat attendu :

```
CALL affichageResultatsRonde('T1', 1);
Triser   Jessica JunF 2385 1-0 Assin    Marc    SenM 1960
Sticko   Judas   SenM 1860 0-1 Némard   Jean    SenM 2235
Alizan   Gaspard SenM 2100 1/2 Bono     Jean    SenM 1850
Kaécouté Xavier  SenM 1850 0-1 Palleja  Xavier  SenM 2080
Nanas    Judas   JunM 2030 1-0 Gator    Ali     JunM 1750

CALL affichageResultatsRonde('T2', 7);
Terrieur Alex    SenM 2455 1-0 Zeblouse Agathe  JunF 2150
Alizan   Gaspard SenM 2100 0-1 Triser   Jessica JunF 2385
Oraque   Anne    VetF 2210 1-0 Némard   Jean    SenM 2235
Terrieur Alain   SenM 2010 1-0 Vesselle Aude    JunF 1940
Kaécouté Xavier  SenM 1850 0-1 Stické   Sophie  JunF 1890
Micoton  Mylène  JunF 1990 1/2 Bono     Jean    SenM 1850
Assin    Marc    SenM 1960 1/2 Outan    Laurent SenM 1920
Nanas    Judas   JunM 2030 1-0 Gator    Ali     JunM 1750
```

Cinquième partie – Edition des résultats d'un joueur à un tournoi :

Dans cette partie, on souhaite pouvoir éditer la fiche des résultats d'un joueur au cours d'un tournoi, avec le nombre total de points qu'il a marqué, son départage (le cumulatif), sa performance Elo, et le classement final qu'il a obtenu. Par exemple, si on édite la fiche des résultats du joueur J4 au tournoi T1 on doit obtenir ce qui suit :

```
Identifiant joueur : J4
Nom joueur : Némard
Prénom joueur : Jean
Elo joueur : 2235
Catégorie joueur : SenM
Club joueur : C1

R1 N Sticko   Judas   SenM 1860  1
R2 B Nanas    Judas   JunM 2030  1
R3 N Palleja  Xavier  SenM 2080  0
R4 B Triser   Jessica JunF 2385 .5
R5 B Alizan   Gaspard SenM 2100 .5

Nombre de points : 3
Cumulatif : 10.5
Perf : 2166
Classement : 3
```

A la ronde 1 (R1), a battu Judas Sticko avec les Noirs

A la ronde 3 (R3), a perdu contre Xavier Palleja avec les Noirs

A la ronde 5 (R5), a fait match nul contre Gaspard Alizan avec les Blancs

Nombre de points obtenus dans le tournoi : 3 (1+1+0+0.5+0.5)

Départage

Performance Elo

Classement final dans le tournoi (3ème)

L'édition de la fiche du joueur J4 au tournoi T2 (où il y avait 7 rondes) doit donner le résultat suivant :

```
Identifiant joueur : J4
Nom joueur : Némard
Prénom joueur : Jean
Elo joueur : 2235
Catégorie joueur : SenM
Club joueur : C1

R1 B Vesselle Aude    JunF 1940  1
R2 N Terrieur Alain   SenM 2010 .5
R3 B Triser   Jessica JunF 2385  0
R4 N Alizan   Gaspard SenM 2100  1
R5 B Zeblouse Agathe  JunF 2150  1
R6 B Terrieur Alex    SenM 2455 .5
R7 N Oraque   Anne    VetF 2210  0

Nombre de points : 4
Cumulatif : 18
Perf : 2232
Classement : 5
```

Si le joueur a terminé invaincu dans le tournoi, cela doit apparaître en toutes lettres dans la fiche des résultats. Par exemple, la joueuse J14 a terminé invaincue dans le tournoi T2 :

```
Identifiant joueur : J14
Nom joueur : Triser
Prénom joueur : Jessica
Elo joueur : 2385
Catégorie joueur : JunF
Club joueur : C1

R1 N Assin    Marc    SenM 1960  1
R2 B Zeblouse Agathe  JunF 2150  1
R3 N Némard   Jean    SenM 2235  1
R4 B Terrieur Alex    SenM 2455 .5
R5 N Oraque   Anne    VetF 2210 .5
R6 B Terrieur Alain   SenM 2010  1
R7 N Alizan   Gaspard SenM 2100  1

Nombre de points : 6
Cumulatif : 24.5
Perf : 2428
Classement : 2 INVAINCU
```

15) Procédure stockée affichageInfosJoueur.

- Ecrire une procédure stockée `affichageInfosJoueur` qui affiche les informations qui concernent un joueur dont l'identifiant `p_idJoueur` est passé en paramètre. Cette procédure doit avoir la signature suivante :

```
PROCEDURE affichageInfosJoueur(
                              p_idJoueur IN Joueurs.idJoueur%TYPE)
```

Si on passe en paramètre de cette procédure un identifiant de joueur qui n'existe pas dans la base de données, un message d'erreur explicite doit être affiché.

- Résultats attendus :

```
CALL affichageInfosJoueur('J10');
Identifiant joueur : J10
Nom joueur : Palleja
Prénom joueur : Xavier
Elo joueur : 2080
Catégorie joueur : SenM
Club joueur : C1

CALL affichageInfosJoueur('J100');
Le joueur J100 n'existe pas
```

16) Fonction stockée resultatEnPoint.

- Ecrire une fonction stockée `resultatEnPoint` qui à partir d'un résultat d'une partie `p_resultat` passé en paramètre (et qui peut avoir comme valeur "1-0", "1/2" ou "0-1"), retourne le nombre de points que marque le joueur qui a la couleur `p_couleur` ("B" ou "N"). Cette fonction doit avoir la signature suivante :

```
FUNCTION resultatEnPoint( p_couleur IN VARCHAR,
                          p_resultat IN Parties.resultatPartie%TYPE)
                                                    RETURN NUMBER
```

Si le résultat est "1-0" c'est les blancs qui ont gagné la partie, le joueur avec les blancs marque alors 1 point et le joueur avec les noirs 0. Si le résultat est "1/2", la partie est déclarée nulle et les deux joueurs marquent 0,5 point. Enfin, si le résultat est "0-1", les noirs ont gagné, le joueur avec les blancs marque 0 point et le joueur avec les noirs 1.

- Résultats attendus :

```
SELECT resultatEnPoint('B', '1-0')
FROM DUAL ;
resultatEnPoint('B', '1-0')
---------------------------
1

SELECT resultatEnPoint('B', '0-1')
FROM DUAL ;
resultatEnPoint('B', '0-1')
---------------------------
0

SELECT resultatEnPoint('N', '1/2')
FROM DUAL ;
resultatEnPoint('N', '1/2')
---------------------------
.5
```

17) Fonction stockée couleurJoueurRonde.

- Ecrire une fonction stockée `couleurJoueurRonde` qui retourne 'B' si le joueur `p_idJoueur` avait les blancs à la ronde `p_numRonde` du tournoi `p_idTournoi`. Elle doit retourner 'N' s'il avait les noirs. La fonction doit retourner `NULL` si le joueur `p_idJoueur` n'a pas participé à la ronde `p_numRonde` du tournoi `p_idTournoi`. Cette fonction doit avoir la signature suivante :

```
FUNCTION couleurJoueurRonde(
                              p_idJoueur IN Joueurs.idJoueur%TYPE,
                              p_idTournoi IN Tournois.idTournoi%TYPE,
                              p_numRonde IN Parties.numRonde%TYPE)
                                                    RETURN VARCHAR
```

- Résultats attendus :

```
SELECT couleurJoueurRonde('J1', 'T1', 1)
FROM DUAL ;

couleurJoueurRonde('J1', 'T1', 1)
----------------------------------
B

SELECT couleurJoueurRonde('J1', 'T1', 2)
FROM DUAL ;

couleurJoueurRonde('J1', 'T1', 2)
----------------------------------
N
```

18) Fonction stockée adversaireJoueurRonde.

- Ecrire une fonction stockée `adversaireJoueurRonde` qui retourne l'identifiant de l'adversaire qu'a rencontré le joueur `p_idJoueur` à la ronde `p_numRonde` du tournoi `p_idTournoi`. La fonction doit retourner `NULL` si le joueur `p_idJoueur` n'a pas participé à la ronde `p_numRonde` du tournoi `p_idTournoi`. Cette fonction doit avoir la signature suivante :

```
FUNCTION adversaireJoueurRonde(
                              p_idJoueur IN Joueurs.idJoueur%TYPE,
                              p_idTournoi IN Tournois.idTournoi%TYPE,
                              p_numRonde IN Parties.numRonde%TYPE)
                                          RETURN Joueurs.idJoueur%TYPE
```

- Résultats attendus :

```
SELECT adversaireJoueurRonde('J10', 'T1', 1)
FROM DUAL ;

adversaireJoueurRonde ('J10', 'T1', 1)
---------------------------------------
J3

SELECT adversaireJoueurRonde('J10', 'T1', 2)
FROM DUAL ;

adversaireJoueurRonde ('J10', 'T1', 2)
---------------------------------------
J14
```

19) Fonction stockée resultatJoueurRonde

Ecrire une fonction stockée `resultatJoueurRonde` qui retourne le nombre de points marqués par le joueur `p_idJoueur` à la ronde `p_numRonde` du tournoi `p_idTournoi`. La fonction doit retourner `NULL` si le joueur `p_idJoueur` n'a pas participé à la ronde `p_numRonde` du tournoi `p_idTournoi`. Cette fonction doit avoir la signature suivante :

```
FUNCTION resultatJoueurRonde(
                             p_idJoueur IN Joueurs.idJoueur%TYPE,
                             p_idTournoi IN Tournois.idTournoi%TYPE,
                             p_numRonde IN Parties.numRonde%TYPE)
                                                    RETURN NUMBER
```

- Résultats attendus :

```
SELECT resultatJoueurRonde('J1', 'T2', 6)
FROM DUAL ;

resultatJoueurRonde ('J1', 'T2', 6)
------------------------------------
1

SELECT resultatJoueurRonde('J1', 'T2', 7)
FROM DUAL ;

resultatJoueurRonde ('J1', 'T2', 7)
------------------------------------
0
```

20) Procédure stockée affichagePartiesJoueurTournoi

- Ecrire une procédure stockée `affichagePartiesJoueurTournoi` qui affiche tous les résultats des parties jouées par le joueur `p_idJoueur` au cours du tournoi `p_idTournoi`. Cette procédure doit avoir la signature suivante :

```
PROCEDURE affichagePartiesJoueurTournoi(
                                p_idJoueur IN Joueurs.idJoueur%TYPE,
                                p_idTournoi IN Tournois.idTournoi%TYPE)
```

- Résultats attendus :

```
CALL affichagePartiesJoueurTournoi ('J4', 'T3') ;

R1 N Bono     Jean    SenM 1850  1
R2 B Kaécouté Xavier  SenM 1850  1
R3 N Terrieur Alex    SenM 2455  0
R4 B Palleja  Xavier  SenM 2080  0
R5 N Zeblouse Agathe  JunF 2150  1

CALL affichagePartiesJoueurTournoi ('J6', 'T2') ;

R1 B Triser   Jessica JunF 2385  0
R2 N Sticé    Sophie  JunF 1890  0
R3 B Bono     Jean    SenM 1850  1
R4 N Vesselle Aude    JunF 1940  0
R5 B Gator    Ali     JunM 1750  1
R6 N Kaécouté Xavier  SenM 1850  0
R7 B Outan    Laurent SenM 1920 .5
```

21) Fonction stockée nbPointsJoueurTournoi.

Ecrire une fonction stockée `nbPointsJoueurTournoi` qui retourne le nombre de points total qu'a obtenu le joueur `p_idJoueur` dans le tournoi `p_idTournoi`. Cette fonction doit avoir la signature suivante :

```
FUNCTION nbPointsJoueurTournoi(
                              p_idJoueur IN Joueurs.idJoueur%TYPE,
                              p_idTournoi IN Tournois.idTournoi%TYPE)
                                                    RETURN NUMBER
```

- Résultats attendus :

```
SELECT nbPointsJoueurTournoi('J10', 'T1')
FROM DUAL ;

nbPointsJoueurTournoi('J10', 'T1')
-----------------------------------
5

SELECT nbPointsJoueurTournoi('J8', 'T1')
FROM DUAL ;

nbPointsJoueurTournoi('J8', 'T1')
-----------------------------------
.5

SELECT nbPointsJoueurTournoi('J8', 'T2')
FROM DUAL ;

nbPointsJoueurTournoi('J8', 'T2')
-----------------------------------
1
```

22) Fonction stockée cumulatifJoueurTournoi.

- Ecrire une fonction stockée `cumulatifJoueurTournoi` qui retourne le départage (c'est-à-dire le cumulatif) que le joueur `p_idJoueur` a obtenu au tournoi `p_idTournoi`. Cette procédure doit avoir la signature suivante :

```
FUNCTION cumulatifJoueurTournoi(
                               p_idJoueur IN Joueurs.idJoueur%TYPE,
                               p_idTournoi IN Tournois.idTournoi%TYPE)
                                                    RETURN NUMBER
```

Pour calculer le cumulatif d'un joueur, il suffit de faire la somme des points qu'il avait après chacune des rondes.

Par exemple, si dans un tournoi à 5 rondes, un joueur a fait 1, 1, 0, ½, 1 ; son cumulatif sera alors de 11 : c'est-à-dire 1 (il avait 1 point après la ronde 1) + 2 (il avait 2 points après la ronde 2) + 2 (il avait toujours 2 points après la ronde 3 – après sa défaite) + 2,5 (il avait 2,5 points après la ronde 4) + 3,5 (il avait 3,5 points après la ronde 5).

- Résultats attendus :

```
SELECT cumulatifJoueurTournoi('J1', 'T2')
FROM DUAL ;

cumulatifJoueurTournoi('J1', 'T2')
-----------------------------------
14.5

SELECT cumulatifJoueurTournoi('J4', 'T1')
FROM DUAL ;

cumulatifJoueurTournoi('J4', 'T1')
-----------------------------------
10.5
```

23) Fonction stockée performanceEloJoueurTournoi.

- Ecrire une fonction stockée `performanceEloJoueurTournoi` qui retourne la performance Elo qu'a obtenue le joueur `p_idJoueur` au tournoi `p_idTournoi`. Cette fonction doit avoir la signature suivante :

```
FUNCTION performanceEloJoueurTournoi(
                            p_idJoueur IN Joueurs.idJoueur%TYPE,
                            p_idTournoi IN TournoiS.idTournoi%TYPE)
                                                    RETURN NUMBER
```

  La performance Elo d'un joueur à un tournoi est calculée en prenant la moyenne Elo des adversaires qu'il a rencontrés dans le tournoi, à quoi on ajoute un bonus/malus qui dépend du score réalisé (ce bonus/malus sera positif si le joueur fait plus de 50% des points et négatif s'il fait moins de 50%). La formule pour calculer ce bonus/malus est : bonus/malus = (pourcentage de points réalisés – 0,5) x 750.

  Par exemple, si dans un tournoi à 5 rondes, un joueur a fait 1 contre un joueur classé 2000, 1 contre un joueur classé 2100, 0 contre un joueur classé 2200, ½ contre un joueur classé 2150, 1 contre un joueur classé 2050 ; La moyenne Elo de ses adversaires est 2100 ; (2000 + 2100 + 2200 + 2050 + 2150) / 5. Comme il a marqué 3,5 points sur 5 son bonus/malus est de +150 ; (3,5/5 -0,5) x 750. Sa performance Elo est donc 2100 + 150 = 2250.

  Si dans le même exemple, il avait perdu la dernière partie, la moyenne Elo de ses adversaires serait inchangée mais son score serait de 2,5/5 et son bonus/malus de 0 ; (2,5/5 – 0,5) x 750. La performance Elo serait alors de 2100 + 0 = 2100.

  Si dans le même exemple, il avait perdu les deux dernières parties, son score serait alors de 2/5 et son bonus/malus de -75 ; (2/5 – 0,5) x 750. La performance Elo serait alors de 2100 - 75 = 2025.

  Il est à noter que la performance Elo est un nombre entier ; il faudra donc l'arrondir le cas échéant. Pour cela, en PL/SQL, vous pourrez utiliser la fonction `ROUND()`.

- Résultats attendus :

```
SELECT performanceEloJoueurTournoi('J26', 'T2')
FROM DUAL ;

performanceEloJoueurTournoi('J26', 'T2')
-----------------------------------------
2405

SELECT performanceEloJoueurTournoi('J11', 'T2')
FROM DUAL ;

performanceEloJoueurTournoi('J11', 'T2')
-----------------------------------------
2168

SELECT performanceEloJoueurTournoi('J8', 'T3')
FROM DUAL ;

performanceEloJoueurTournoi('J8', 'T3')
---------------------------------------
2030

SELECT performanceEloJoueurTournoi('J24', 'T2')
FROM DUAL ;

performanceEloJoueurTournoi('J24', 'T2')
-----------------------------------------
1761
```

24) Procédure stockée affichageResultsJoueurTournoi.

- Ecrire une procédure stockée `affichageResultsJoueurTournoi` qui affiche les résultats obtenus par le joueur `p_idJoueur` au tournoi `p_idTournoi`. On veut que soient affichés le nombre de points obtenus par le joueur ainsi que son cumulatif et sa performance Elo. La procédure doit avoir la signature suivante :

```
PROCEDURE affichageResultsJoueurTournoi(
                                p_idJoueur IN Joueurs.idJoueur%TYPE,
                                p_idTournoi IN Tournois.idTournoi%TYPE)
```

- Résultats attendus :

```
CALL affichageResultsJoueurTournoi('J1', 'T2') ;

Nombre de points : 3.5
Cumulatif : 14.5
Perf : 2114

CALL affichageResultsJoueurTournoi('J10', 'T3') ;

Nombre de points : 4
Cumulatif : 10.5
Perf : 2281
```

25) Fonction stockée estInvaincu

- Ecrire une fonction stockée `estInvaincu` qui retourne `'1'` si le joueur `p_idJoueur` a terminé invaincu dans le tournoi `p_idTournoi ; '0'` sinon. Un joueur invaincu est un joueur qui n'a pas perdu de partie dans le tournoi (mais il a pu éventuellement faire des parties nulles). Cette fonction doit avoir la signature suivante :

```
FUNCTION estInvaincu(
                    p_idJoueur IN Joueurs.idJoueur%TYPE,
                    p_idTournoi IN TournoiS.idTournoi%TYPE)
                                                    RETURN NUMBER
```

- Résultats attendus :

```
SELECT estInvaincu('J14', 'T2')
FROM DUAL ;

estInvaincu('J14', 'T2')
-------------------------
1

SELECT estInvaincu('J27', 'T2')
FROM DUAL ;

estInvaincu('J27', 'T2')
-------------------------
0
```

26) Fonction stockée classementJoueurTournoi

- Ecrire une fonction stockée `classementJoueurTournoi` qui retourne le classement obtenu par le joueur `p_idJoueur` dans le tournoi `p_idTournoi`.
  Cette fonction doit avoir la signature suivante :

```
FUNCTION classementJoueurTournoi(
                                p_idJoueur IN Joueurs.idJoueur%TYPE,
                                p_idTournoi IN Tournois.idTournoi%TYPE)
                                                    RETURN NUMBER
```

  Le classement est réalisé à partir du nombre de points qu'ont obtenu les différents joueurs dans le tournoi. Si deux joueurs (ou plus) ont le même nombre de points, on les départage avec le cumulatif. Si ils ont le même cumulatif, on regarde alors la performance Elo qu'ils ont réalisée. S'ils ont aussi la même performance Elo, ils sont départagés par leur propre Elo (celui qui a le plus gros Elo termine devant). Enfin, si ils ne sont toujours pas départagés avec les critères précédents, ils sont classés par ordre alphabétique de leur nom.

- Résultats attendus :

```
SELECT classementJoueurTournoi('J10', 'T1')
FROM DUAL ;

classementJoueurTournoi('J10', 'T1')
------------------------------------
1

SELECT classementJoueurTournoi('J23', 'T1')
FROM DUAL ;

classementJoueurTournoi('J23', 'T1')
------------------------------------
5

SELECT classementJoueurTournoi('J3', 'T2')
FROM DUAL ;

classementJoueurTournoi('J3', 'T2')
-----------------------------------
14
```

27) Procédure stockée affichageClasmntJoueurTournoi.

- Ecrire une procédure stockée `affichageClasmntJoueurTournoi` qui affiche le classement d'un joueur `p_idJoueur` au tournoi `p_idTournoi` et qui indique également s'il a été invaincu. Cette procédure doit avoir la signature suivante :

```
PROCEDURE affichageClasmntJoueurTournoi(
                                p_idJoueur IN Joueurs.idJoueur%TYPE,
                                p_idTournoi IN Tournois.idTournoi%TYPE)
```

- Résultats attendus :

```
CALL affichageClasmntJoueurTournoi('J14', 'T2') ;

Classement : 2 INVAINCU


CALL affichageClasmntJoueurTournoi('J26', 'T3') ;

Classement : 1
```

28) Procédure stockée affichageFicheJoueurTournoi.

- Ecrire une procédure stockée `affichageFicheJoueurTournoi` qui affiche la fiche des résultats du joueur `p_idJoueur` au tournoi `p_idTournoi`. Cette procédure doit avoir la signature suivante :

```
PROCEDURE affichageFicheJoueurTournoi(
                                p_idJoueur IN Joueurs.idJoueur%TYPE,
                                p_idTournoi IN Tournois.idTournoi%TYPE)
```

- Résultats attendus :

```
CALL affichageFicheJoueurTournoi('J26', 'T3') ;

Identifiant joueur : J26
Nom joueur : Terrieur
Prénom joueur : Alex
Elo joueur : 2455
Catégorie joueur : SenM
Club joueur : C4

R1 B Timettre  Vincent JunM 1850  1
R2 N Zetofrais Mélanie SenF 2080  1
R3 B Némard    Jean    SenM 2235  1
R4 N Ouzy      Jacques JunM 1840  1
R5 B Palleja   Xavier  SenM 2080  0

Nombre de points : 4
Cumulatif : 14
Perf : 2242
Classement : 1
```

```
CALL affichageFicheJoueurTournoi('J11', 'T2') ;

Identifiant joueur : J11
Nom joueur : Zeblouse
Prénom joueur : Agathe
Elo joueur : 2150
Catégorie joueur : JunF
Club joueur : C8

R1 B Stické   Sophie  JunF 1890  1
R2 N Triser   Jessica JunF 2385  0
R3 B Terrieur Alain   SenM 2010  1
R4 N Oraque   Anne    VetF 2210 .5
R5 N Némard   Jean    SenM 2235  0
R6 B Micoton  Mylène  JunF 1990  1
R7 N Terrieur Alex    SenM 2455  0

Nombre de points : 3.5
Cumulatif : 16
Perf : 2168
Classement : 7



CALL affichageFicheJoueurTournoi('J14', 'T2') ;

Identifiant joueur : J14
Nom joueur : Triser
Prénom joueur : Jessica
Elo joueur : 2385
Catégorie joueur : JunF
Club joueur : C1

R1 N Assin    Marc    SenM 1960  1
R2 B Zeblouse Agathe  JunF 2150  1
R3 N Némard   Jean    SenM 2235  1
R4 B Terrieur Alex    SenM 2455 .5
R5 N Oraque   Anne    VetF 2210 .5
R6 B Terrieur Alain   SenM 2010  1
R7 N Alizan   Gaspard SenM 2100  1

Nombre de points : 6
Cumulatif : 24.5
Perf : 2428
Classement : 2 INVAINCU



CALL affichageFicheJoueurTournoi('J14', 'T1') ;

Identifiant joueur : J14
Nom joueur : Triser
Prénom joueur : Jessica
Elo joueur : 2385
Catégorie joueur : JunF
Club joueur : C1

R1 B Assin    Marc    SenM 1960  1
R2 N Palleja  Xavier  SenM 2080  0
R3 B Sticko   Judas   SenM 1860  1
R4 N Némard   Jean    SenM 2235 .5
R5 B Nanas    Judas   JunM 2030  1

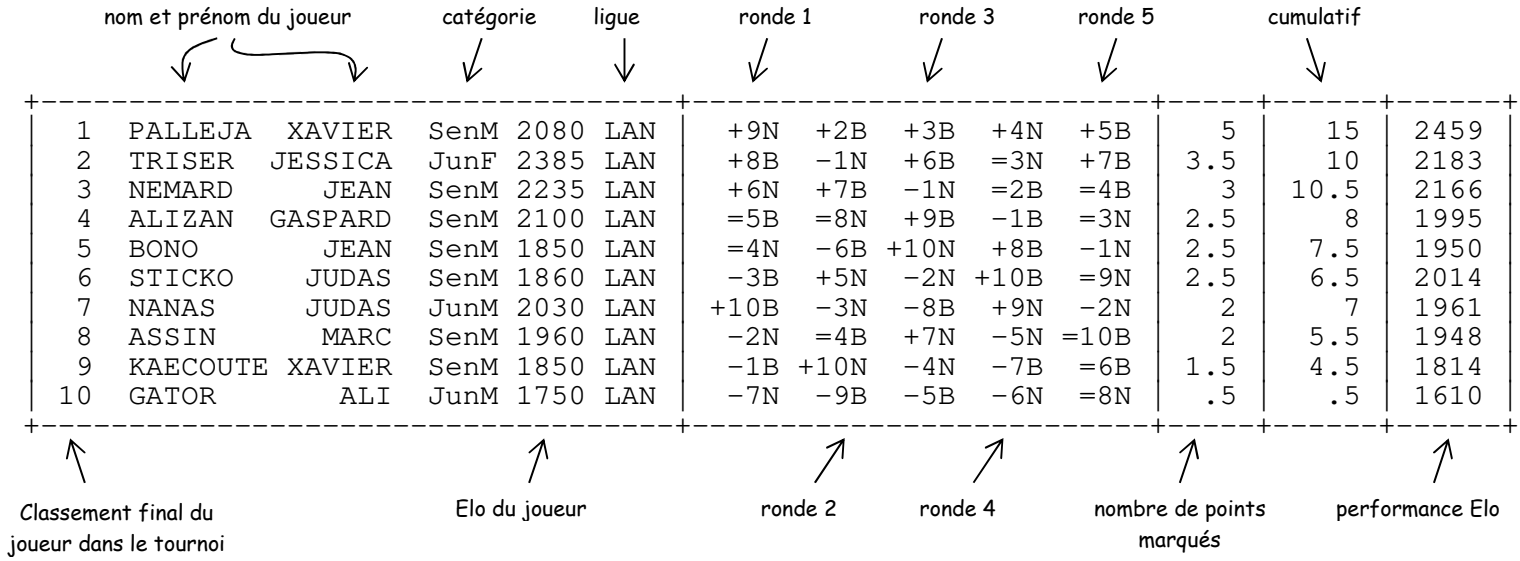
Nombre de points : 3.5
Cumulatif : 10
Perf : 2183
Classement : 2
```

Sixième partie – Edition de la grille américaine d'un tournoi :

Dans cette dernière partie on souhaite éditer le classement complet d'un tournoi sous la forme d'un tableau que l'on appelle, dans les milieux autorisés, "grille américaine".

Sur une grille américaine, les joueurs sont numérotés et triés par rapport au classement qu'ils ont obtenu dans le tournoi (le joueur 1 est celui qui a terminé premier, le joueur 2 celui qui a fini deuxième, etc.). Chacun des joueurs est décrit par son nom, son prénom, sa catégorie, son Elo et sa ligue régionale. La grille américaine permet de suivre, ronde par ronde, le parcours qu'a réalisé le joueur pendant le tournoi ; avec pour chacune de ses parties, l'adversaire qu'il a rencontré, la couleur qu'il avait (`B` ou `N`), et le résultat (`+ = −`). Par exemple, `+5B` signifie qu'il a battu avec les blancs le joueur qui a terminé cinquième. `−4N`, signifie qu'il a perdu avec les noirs contre le joueur qui a terminé quatrième. `=3B`, signifie qu'il a fait match nul avec les blancs contre le joueur qui a terminé troisième. Enfin, la grille américaine indique aussi pour chacun des joueurs du tournoi, le nombre de points total qu'il a marqué, son départage (le cumulatif) et sa performance Elo.

Par exemple, la grille américaine du tournoi T1 donnerait le tableau suivant :



1	PALLEJA	XAVIER	SenM	2080	LAN	+9N	+2B	+3B	+4N	+5B	5	15	2459
2	TRISER	JESSICA	JunF	2385	LAN	+8B	−1N	+6B	=3N	+7B	3.5	10	2183
3	NEMARD	JEAN	SenM	2235	LAN	+6N	+7B	−1N	=2B	=4B	3	10.5	2166
4	ALIZAN	GASPARD	SenM	2100	LAN	=5B	=8N	+9B	−1B	=3N	2.5	8	1995
5	BONO	JEAN	SenM	1850	LAN	=4N	−6B	+10N	+8B	−1N	2.5	7.5	1950
6	STICKO	JUDAS	SenM	1860	LAN	−3B	+5N	−2N	+10B	=9N	2.5	6.5	2014
7	NANAS	JUDAS	JunM	2030	LAN	+10B	−3N	−8B	+9N	−2N	2	7	1961
8	ASSIN	MARC	SenM	1960	LAN	−2N	=4B	+7N	−5N	=10B	2	5.5	1948
9	KAECOUTE	XAVIER	SenM	1850	LAN	−1B	+10N	−4N	−7B	=6B	1.5	4.5	1814
10	GATOR	ALI	JunM	1750	LAN	−7N	−9B	−5B	−6N	=8N	.5	.5	1610

Dans l'exemple précédent, le joueur Jean Bono qui a terminé cinquième du tournoi, a marqué au total 2,5 points sur 5 parties, a un cumulatif de 7,5 et sa performance Elo sur le tournoi est de 1950. Lors de la première ronde, il a fait match nul avec les noirs contre le joueur qui a terminé quatrième (Gaspard Alizan). Lors de la ronde 2, il a perdu avec les blancs contre le joueur qui a terminé sixième (Judas Sticko). Lors de la ronde 3, il a battu avec les noirs le joueur qui a terminé dixième (Ali Gator). Lors de la ronde 4, il a battu avec les blancs le joueur qui a terminé huitième (Marc Assin). Puis lors de la dernière ronde, il a perdu avec les noirs contre le joueur qui a terminé premier (Xavier Palleja).

On veut que le nom et le prénom des joueurs soient écrits en lettres majuscules. Si un nom ou un prénom fait plus de 10 lettres, il pourra être tronqué.

29) Procédure stockée affichageGrilleAmericiane.

- Ecrire une procédure stockée `affichageGrilleAmericaine` qui affiche la grille américaine d'un tournoi dont l'identifiant `p_idTournoi` est passé en paramètre. Cette procédure doit avoir la signature suivante :

```
PROCEDURE affichageGrilleAmericaine(
                                p_idTournoi IN Tournois.idTournoi%TYPE)
```

  Pour améliorer la lisibilité de votre code et faciliter sa maintenance, il est fortement conseillé de créer d'autres procédures stockées qui seront appelées à l'intérieur du corps de la procédure `affichageGrilleAmericaine`.

  Afin d'améliorer les performances, vous avez la possibilité, si vous le souhaitez, de rajouter des champs calculés dans votre Schéma Relationnel.

  Pour mettre en forme votre texte, vous pourrez utiliser les fonctions Oracle suivantes :
  - `LPAD()` et `RPAD()` qui permettent d'aligner un texte sur la gauche ou sur la droite et qui permet également de le tronquer si il est trop long.
  - `UPPER()` qui permet de mettre en majuscules toutes les lettres d'un texte.
  - `TRANSLATE()` qui peut être utilisé pour remplacer les lettres accentuées d'un texte par des lettres non accentuées (par exemple, remplacer É par E).

→ Il est à noter qui si on affiche le résultat directement dans le navigateur, on risque de perdre la mise en forme (les espaces consécutifs vont disparaître). Pour cette raison, demandez à iSQL*Plus de ne pas générer le résultat dans le navigateur mais dans un fichier externe que vous pourrez ouvrir avec votre éditeur de texte préféré (par exemple Notepad++) :
Préférences – Emplacement de sortie – Enregistrer dans un fichier HTML.

Comme votre fichier externe sera ouvert avec un éditeur texte, demandez également à iSQL*Plus, lors de la génération du fichier, de ne pas remplacer les sauts de lignes par des balises `<BR>` :
Préférences – Formatage du script – Sortie préformatée – Activer (On).

- Résultats attendus : voir pages suivantes.

```
CALL affichageGrilleAmericaine('T1');
```

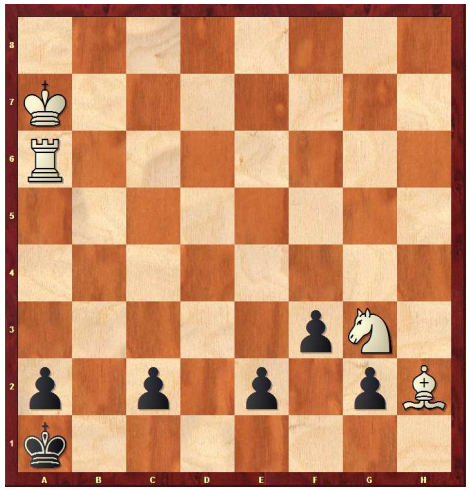
1	PALLEJA	XAVIER	SenM	2080	LAN	+9N	+2B	+3B	+4N	+5B	5	15	2459
2	TRISER	JESSICA	JunF	2385	LAN	+8B	−1N	+6B	=3N	+7B	3.5	10	2183
3	NEMARD	JEAN	SenM	2235	LAN	+6N	+7B	−1N	=2B	=4B	3	10.5	2166
4	ALIZAN	GASPARD	SenM	2100	LAN	=5B	=8N	+9B	−1B	=3N	2.5	8	1995
5	BONO	JEAN	SenM	1850	LAN	=4N	−6B	+10N	+8B	−1N	2.5	7.5	1950
6	STICKO	JUDAS	SenM	1860	LAN	−3B	+5N	−2N	+10B	=9N	2.5	6.5	2014
7	NANAS	JUDAS	JunM	2030	LAN	+10B	−3N	−8B	+9N	−2N	2	7	1961
8	ASSIN	MARC	SenM	1960	LAN	−2N	=4B	+7N	−5N	=10B	2	5.5	1948
9	KAECOUTE	XAVIER	SenM	1850	LAN	−1B	+10N	−4N	−7B	=6B	1.5	4.5	1814
10	GATOR	ALI	JunM	1750	LAN	−7N	−9B	−5B	−6N	=8N	.5	.5	1610

```
CALL affichageGrilleAmericaine('T2');
```

1	TERRIEUR	ALEX	SenM	2455	IDF	+10B	+8N	+3B	=2N	+6B	=5N	+7B	6	25	2405
2	TRISER	JESSICA	JunF	2385	LAN	+13N	+7B	+5N	=1B	=3N	+4B	+8N	6	24.5	2428
3	ORAQUE	ANNE	VetF	2210	IDF	+15N	+9B	−1N	=7B	=2B	+6N	+5B	5	19.5	2313
4	TERRIEUR	ALAIN	SenM	2010	HNO	+16N	=5B	−7N	+10B	+9N	−2N	+11B	4.5	18	2176
5	NEMARD	JEAN	SenM	2235	LAN	+11B	=4N	−2B	+8N	+7B	=1B	−3N	4	18	2232
6	STICKE	SOPHIE	JunF	1890	MPY	−7N	+13B	+15N	+9B	−1N	−3B	+14N	4	16	2136
7	ZEBLOUSE	AGATHE	JunF	2150	AQU	+6B	−2N	+4B	=3N	−5N	+10B	−1N	3.5	16	2168
8	ALIZAN	GASPARD	SenM	2100	LAN	+14N	−1B	=10N	−5B	+12N	+9B	−2B	3.5	14.5	2114
9	NANAS	JUDAS	JunM	2030	LAN	+12B	−3N	+11B	−6N	−4B	−8N	+16B	3	13	1911
10	MICOTON	MYLENE	JunF	1990	IDF	−1N	+14B	=8B	−4N	+11B	−7N	=12B	3	12	1997
11	VESSELLE	AUDE	JunF	1940	MPY	−5N	+16B	−9N	+13B	−10N	+15B	−4N	3	12	1931
12	BONO	JEAN	SenM	1850	LAN	−9N	=15N	−13N	+14B	−8B	=16N	=10N	2.5	8.5	1836
13	ASSIN	MARC	SenM	1960	LAN	−2B	−6N	+12B	−11N	+16B	−14N	=15B	2.5	8.5	1834
14	KAECOUTE	XAVIER	SenM	1850	LAN	−8B	−10N	+16B	−12N	=15N	+13B	−6B	2.5	8.5	1816
15	OUTAN	LAURENT	SenM	1920	AQU	−3B	=12B	−6B	=16N	=14B	−11N	=13N	2	7	1761
16	GATOR	ALI	JunM	1750	LAN	−4B	−11N	−14N	=15B	−13N	=12B	−9N	1	3	1669

```
CALL affichageGrilleAmericaine('T3');
```

1	TERRIEUR	ALEX	SenM	2455	IDF	+12B	+3N	+4B	+5N	-2B	4	14	2242
2	PALLEJA	XAVIER	SenM	2080	LAN	=8B	=12N	+11B	+4N	+1N	4	10.5	2281
3	ZETOFRAIS	MELANIE	SenF	2080	PRO	+10N	-1B	+9N	=7B	+8N	3.5	10	2147
4	NEMARD	JEAN	SenM	2235	LAN	+6N	+9B	-1N	-2B	+7N	3	10	2152
5	OUZY	JACQUES	JunM	1840	MPY	-7N	+13B	+10N	-1B	+14N	3	8	1980
6	BONO	JEAN	SenM	1850	LAN	-4B	-8N	+12B	+13B	+11N	3	6	1978
7	ZEBLOUSE	AGATHE	JunF	2150	AQU	+5B	=11N	=8B	=3N	-4B	2.5	9.5	1979
8	GATOR	ALI	JunM	1750	LAN	=2N	+6B	=7N	=11B	-3B	2.5	9	2030
9	KAECOUTE	XAVIER	SenM	1850	LAN	+14B	-4N	-3B	=10N	+12B	2.5	7	1879
10	BRICOT	JUDAS	VetM	1780	LAN	-3B	+14N	-5B	=9B	+13N	2.5	6	1782
11	MICOTON	MYLENE	JunF	1990	IDF	+13N	=7B	-2N	=8N	-6B	2	8	1829
12	TIMETTRE	VINCENT	JunM	1850	IDF	-1N	=2B	-6N	+14B	-9N	1.5	4	1787
13	CROQUE	ODILE	VetF	1690	LAN	-11B	-5N	+14B	-6N	-10B	1	3	1557
14	JAVEL	AUDE	VetF	1450	LAN	-9N	-10B	-13N	-12N	-5B	0	0	1427



Les blancs jouent et font mat en 18 coups